МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«СИБИРСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ТЕЛЕКОММУНИКАЦИЙ И ИНФОРМАТИКИ» (СибГУТИ)
Институт вычислительной техники
Кафедра вычислительных систем

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ № 6

по дисциплине «Основы разработки на Java»

Выполнил:
студент гр. ИС-541
Устюжанин Д. К.

Проверил:
доцент кафедры ВС
Нигматулин Р. Р.

Новосибирск, 2026

Содержание

1. Введение 3
1.1 Цель работы 3
1.2 Предметная область и стек технологий 3
2. Раздел I. Строки 4
2.1 Задание 1. String: identity и value 4
2.2 Задание 2. Сборка строки 5
2.3 Задание 3. Регулярное выражение с группами 5
3. Раздел II. Числа 8
3.1 Задание 4. Autoboxing и null 8
3.2 Задание 5. BigDecimal без сюрприза double 8
3.3 Задание 6. Масштаб и округление 9
3.4 Задание 7. Math и воспроизводимая случайность 10
4. Раздел III. Дата и время 11
4.1 Задание 8. LocalDate и календарные правила 11
4.2 Задание 9. Period против Duration 11
4.3 Задание 10. Строгий DateTimeFormatter 12
4.4 Задание 11. Instant и пользовательская зона 13
4.5 Задание 12. DST-кейс 13
5. Раздел IV. Проектирование домена 15
5.1 Задание 13. Нормализованный поиск 15
5.2 Задание 14. Money и цена курса 15
5.3 Задание 15. Расписание и цена вокруг границ времени 16
6. Заключение 18
Приложение A. Полные листинги программы 19

1. Введение

1.1 Цель работы

Освоить базовые типы из «повседневного» арсенала Java и их подводные камни: ссылочность и неизменяемость String, регулярные выражения с группами, автоупаковку и null, точные вычисления через BigDecimal, масштаб и режимы округления, воспроизводимую случайность, а также новый API времени (LocalDate, Period, Duration, Instant, ZonedDateTime, DateTimeFormatter). Завершается работа сквозной задачей цен и расписания вебинаров CourseHub, где все эти типы применяются совместно.

1.2 Предметная область и стек технологий

Сквозная тема — учебный центр CourseHub: коды курсов, названия, цены, расписание вебинаров, скидка early-bird и возвраты.

- **Предметная модель:** Система управления учебным центром CourseHub (курсы, зачисления, вебинары, ценообразование).
- **Стек технологий:** Java 21, Maven, JUnit 5.

2. Раздел I. Строки

2.1 Задание 1. String: identity и value

Постановка задачи:
Предсказать == и equals для литералов, new String, конкатенации compile-time и runtime, а также intern. Сформулировать правило сравнения текста, не опирающееся на string pool.

Анализ решения и теоретическое обоснование:
Оператор == для ссылочных типов сравнивает идентичность (ссылку), а equals — содержимое. String pool — это кэш литералов и строк, прошедших intern(). Компилятор сворачивает константные выражения во время компиляции. Конкатенация с участием переменной выполняется в runtime и создает новый объект вне pool. Правило: текст сравниваем через equals (или Objects.equals), никогда через ==.

Код решения:

Листинг 1 — *Сравнение ссылочной идентичности и значений строк*

```
String a = "JAVA-101";
String b = "JAVA-101";
System.out.println("literal == literal: " + (a == b)); // true (pool)
String c = new String("JAVA-101");
System.out.println("literal == new String: " + (a == c)); // false
String d = "JAVA" + "-101";
System.out.println("literal == compile-time concat: " + (a == d)); //
true
String prefix = "JAVA";
String e = prefix + "-101";
System.out.println("literal == runtime concat: " + (a == e)); // false
System.out.println("literal == interned: " + (a == e.intern())); // true
```

Выводы по заданию:
Литералы и compile-time конкатенации используют string pool. Runtime-конкатенация и new String создают новые объекты в куче. Сравнение строк необходимо всегда выполнять через equals().

2.2 Задание 2. Сборка строки

Постановка задачи: Сформировать отчёт из 10 000 строк через + в цикле и через StringBuilder. Сравнить результат и время работы. Объяснить неизменяемость String.

Анализ решения и теоретическое обоснование: String неизменяем (immutable): любой оператор + создает новый объект String. В цикле операция report += line копирует все накопленные данные, приводя к сложности $O(n^2)$. StringBuilder использует изменяемый буфер и дописывает элементы за амортизированное время O(1), обеспечивая линейную сложность O(n).

Код решения:

Листинг 2 — *Сравнение сборки строки через + и StringBuilder*

```
public static String withPlus(List<String> lines) {
    String report = "";
    for (String line : lines) {
        report += line + "\n";
    }
    return report;
}

public static String withBuilder(List<String> lines) {
    StringBuilder sb = new StringBuilder();
    for (String line : lines) {
        sb.append(line).append('\n');
    }
    return sb.toString();
}
```

Выводы по заданию: Результаты обеих реализаций побайтово совпадают, однако StringBuilder работает значительно быстрее на больших объемах данных благодаря отсутствию постоянного пересоздания объектов.

2.3 Задание 3. Регулярное выражение с группами

Постановка задачи: Проверить код курса вида JAVA-101, извлечь prefix и number, запретить

частичное совпадение. Подготовить таблицу валидных и невалидных значений.

Анализ решения и теоретическое обоснование: Регулярное выражение \A([A-Z]+)-(\d+)\z использует якоря \A и \z для полного совпадения всей строки и две захватывающие группы (...) для извлечения компонентов.

Таблица 1 — Валидация кодов курсов

№	Значение	Результат	Обоснование
1	`JAVA-101`	valid (JAVA, 101)	Полное соответствие шаблону
2	`GO-7`	valid (GO, 7)	Валидный префикс и номер
3	`JAVA-101x`	invalid	Частичное совпадение отклоняется
4	`java-101`	invalid	Строчные буквы запрещены шаблоном [A-Z]
5	`ЯВА-101`	invalid	Кириллица отклоняется

Код решения:

Листинг 3 — *Парсинг кода курса регулярным выражением*

```
private static final Pattern FORMAT = Pattern.compile("\\A([A-Z]+)-
(\\d+)\\z");

public static Optional<CourseCode> parse(String raw) {
    if (raw == null) return Optional.empty();
    Matcher matcher = FORMAT.matcher(raw);
    if (!matcher.matches()) return Optional.empty();
    return Optional.of(new CourseCode(matcher.group(1),
Integer.parseInt(matcher.group(2))));
}
```

Выводы по заданию:

Применение строгого регулярного выражения с якорями гарантирует корректную валидацию и исключает частичные совпадения и некорректные символы.

3. Раздел II. Числа

3.1 Задание 4. Autoboxing и null

Постановка задачи:
Воспроизвести NPE при unboxing Integer, сравнить Integer через == на разных значениях и исправить код через явный контракт.

Анализ решения и теоретическое обоснование:
Распаковка (unboxing) значения null приводит к NullPointerException. JVM кэширует объекты Integer в диапазоне [-128..127], поэтому == дает true в этом диапазоне и false за его пределами. Для корректной работы необходимо использовать Objects.equals().

Код решения:

Листинг 4 — *Безопасная обработка Integer и проверка равенства*

```
public static int plusOne(Integer value) {
    return value == null ? 0 : value + 1;
}

public static boolean sameByValue(Integer left, Integer right) {
    return java.util.Objects.equals(left, right);
}
```

Выводы по заданию:
Обработка null должна определяться явным контрактом, а сравнение оберточных типов следует производить исключительно по значению через equals().

3.2 Задание 5. BigDecimal без сюрприза double

Постановка задачи:
Сравнить new BigDecimal(0.1), BigDecimal.valueOf(0.1) и строковый конструктор. Рассчитать стоимость трех курсов без использования double.

Анализ решения и теоретическое обоснование:
Тип double хранит значение в двоичной системе счисления, где 0.1 является

бесконечной дробью. Конструктор new BigDecimal(double) точно передает это двоичное представление (0.1000000000000000055511...). BigDecimal.valueOf(0.1) и new BigDecimal("0.1") дают точное десятичное значение.

Код решения:

Листинг 5 — *Инициализация и сложение точных десятичных чисел*

```
BigDecimal fromDouble = new BigDecimal(0.1);
BigDecimal valueOf = BigDecimal.valueOf(0.1);
BigDecimal fromString = new BigDecimal("0.1");
BigDecimal total = new
BigDecimal("3000.00").add(BigDecimal.valueOf(1500.50)).add(new
BigDecimal("999.99"));
```

Выводы по заданию:

Для денежных расчетов следует использовать исключительно строковый конструктор или метод BigDecimal.valueOf().

3.3 Задание 6. Масштаб и округление

Постановка задачи:

Распределить скидку между позициями так, чтобы итоговая сумма точно совпала с ожидаемой. Остаток копеек раздать детерминированно.

Анализ решения и теоретическое обоснование:

При делении денежных сумм возникает остаток (копейки), потеря которого недопустима. Вычисление пропорций выполняется с явным RoundingMode, а нераспределенный остаток поочередно добавляется к позициям по кругу.

Код решения:

Листинг 6 — *Распределение скидки без потери копеек*

```
BigDecimal share =
amount.multiply(BigDecimal.valueOf(weights[i])).divide(BigDecimal.valueO
f(totalWeight), SCALE, mode);
BigDecimal remainder = amount.setScale(SCALE,
mode).subtract(distributed);
while (remainder.signum() != 0) {
    BigDecimal step = remainder.signum() > 0 ? CENT : CENT.negate();
    shares[index] = shares[index].add(step);
```

```
    remainder = remainder.subtract(step);
    index = (index + 1) % shares.length;
}
```

Выводы по заданию:

Алгоритм детерминированно раздает остаток, благодаря чему сумма частей строго равняется исходной величине.

3.4 Задание 7. Math и воспроизводимая случайность

Постановка задачи:

Сгенерировать выборку через RandomGenerator с seed, вычислить границы и среднее с явным округлением.

Анализ решения и теоретическое обоснование:

Интерфейс RandomGenerator (Java 17+) позволяет отделить генерацию случайных чисел в продакшене от воспроизводимых тестов с фиксированным seed. Среднее значение рассчитывается в BigDecimal с режимом RoundingMode.HALF_UP.

Код решения:

Листинг 7 — *Воспроизводимая случайность и расчет статистики*

```
public static RandomGenerator seeded(long seed) {
    return RandomGeneratorFactory.of("L64X128MixRandom").create(seed);
}
BigDecimal average =
BigDecimal.valueOf(sum).divide(BigDecimal.valueOf(values.length), 2,
RoundingMode.HALF_UP);
```

Выводы по заданию:

Использование фиксированного seed гарантирует повторяемость тестов, а точное округление предохраняет от потерь при делении.

4. Раздел III. Дата и время

4.1 Задание 8. LocalDate и календарные правила

Постановка задачи:
Рассчитать даты начала модулей, последний учебный день и проверить високосный год. Исходный объект даты не должен изменяться.

Анализ решения и теоретическое обоснование:
Класс LocalDate неизменяем (immutable). Все методы типа plusWeeks() или minusDays() возвращают новый экземпляр, который необходимо присваивать переменной.

Код решения:

Листинг 8 — *Расчет учебных модулей на LocalDate*

```
LocalDate date = start;
for (int i = 0; i < modules; i++) {
    starts.add(date);
    date = date.plusWeeks(moduleWeeks);
}
boolean isLeap = Year.isLeap(start.getYear());
```

Выводы по заданию:
Продемонстрирована неизменяемость LocalDate — исходный объект даты остался неизменным после всех календарных вычислений.

4.2 Задание 9. Period против Duration

Постановка задачи:
Сравнить «календарный день» и «24 часа» при переходе на летнее время (DST).

Анализ решения и теоретическое обоснование:
Period работает с календарными единицами (дни, месяцы), а Duration — с точным физическим временем (секунды, часы). В день перехода на летнее время календарный день содержит 23 часа, из-за чего Period.ofDays(1) и Duration.ofHours(24) дают разные результаты.

Код решения:

Листинг 9 — *Сравнение Period и Duration при сдвиге времени*

```
public ZonedDateTime plusCalendarDays(int days, ZoneId zone) {
    return start.atZone(zone).plus(Period.ofDays(days));
}
public Instant plusExactHours(long hours) {
    return start.plus(Duration.ofHours(hours));
}
```

Выводы по заданию:

Для бизнес-сроков (например, подписки) следует применять Period, а для таймаутов и физических интервалов — Duration.

4.3 Задание 10. Строгий DateTimeFormatter

Постановка задачи:

Парсить даты формата dd.MM.uuuu в строгом режиме, отклоняя невозможные даты и лишний текст.

Анализ решения и теоретическое обоснование:

ResolverStyle.STRICT в сочетании с символом года uuuu запрещает автоматическую коррекцию некорректных дат (например, 31 февраля) и требует полного соответствия строке без хвостовых символов.

Код решения:

Листинг 10 — *Строгий парсер дат*

```
private static final DateTimeFormatter FORMAT =
DateTimeFormatter.ofPattern("dd.MM.uuuu").withResolverStyle(ResolverStyl
e.STRICT);

public static LocalDate parse(String raw) {
    try {
        return LocalDate.parse(raw, FORMAT);
    } catch (DateTimeParseException e) {
        throw new InvalidDateFieldException("date", raw, e);
    }
}
```

Выводы по заданию:

Использование STRICT позволяет надежно валидировать ввод

пользователя и выбрасывать понятные исключения с сохранением первопричины (cause).

4.4 Задание 11. Instant и пользовательская зона

Постановка задачи:
Хранить момент вебинара как Instant и отображать в нескольких часовых поясах.

Анализ решения и теоретическое обоснование:
Instant представляет собой точную точку на временной шкале UTC. Часовые пояса (ZoneId) должны применяться только при отображении пользователю, чтобы бизнес-логика не зависела от системной часовой зоны сервера.

Код решения:

Листинг 11 — *Отображение Instant в различных временных зонах*

```
public ZonedDateTime displayIn(ZoneId zone) {
    return start.atZone(zone);
}
// Смена зоны сервера не влияет на Instant
TimeZone.setDefault(TimeZone.getTimeZone("Asia/Tokyo"));
```

Выводы по заданию:
Модель данных остается стабильной независимо от настроек операционной системы или сервера.

4.5 Задание 12. DST-кейс

Постановка задачи:
Обработать пропущенное и повторенное местное время при переходе на летнее/зимнее время.

Анализ решения и теоретическое обоснование:
При весеннем переходе час времени пропадает (gap), а при осеннем — повторяется (overlap). Стандартный метод atZone() сдвигает

несуществующее время вперед, однако строгая политика должна выбрасывать исключение InvalidLocalTimeException.

Код решения:

Листинг 12 — *Политика разрешения несуществующего и повторяющегося времени*

```
public static ZonedDateTime resolveStrict(LocalDateTime local, ZoneId
zone) {
    ZonedDateTime resolved = local.atZone(zone);
    if (!resolved.toLocalDateTime().equals(local)) {
        throw new InvalidLocalTimeException(local, zone);
    }
    return resolved;
}
```

Выводы по заданию:

Явная политика обработки DST исключает непредсказуемое поведение расписания при сдвигах часов.

5. Раздел IV. Проектирование домена

5.1 Задание 13. Нормализованный поиск

Постановка задачи: Реализовать конвейер нормализации названий курсов и отделить отображаемое значение от поискового ключа.

Анализ решения и теоретическое обоснование: Конвейер нормализации включает Unicode NFKC, обрезку пробелов, схлопывание внутренних пробелов и приведение к нижнему регистру с фиксацией Locale.ROOT. Это позволяет находить записи независимо от регистра и форматов ввода.

Код решения:

Листинг 13 — *Формирование поискового ключа CourseTitle*

```
public static String searchKeyOf(String raw) {
    return Normalizer.normalize(raw, Normalizer.Form.NFKC)
        .strip()
        .replaceAll("\\s+", " ")
        .toLowerCase(Locale.ROOT);
}
```

Выводы по заданию: Разделение display и searchKey обеспечивает корректный поиск и отображает коллизии при совпадении ключей.

5.2 Задание 14. Money и цена курса

Постановка задачи: Спроектировать неизменяемый тип Money с запретом смешения валют, точным округлением и распределением долей.

Анализ решения и теоретическое обоснование: Класс Money объединяет BigDecimal и Currency. Он гарантирует неизменяемость, валидирует совпадение валют при операциях и реализует equals() через compareTo(), а hashCode() через stripTrailingZeros().

Код решения:

Листинг 14 — *Неизменяемый денежный класс Money*

```
public Money add(Money other) {
    requireSameCurrency(other);
    return new Money(amount.add(other.amount), currency);
}

public Money percentage(BigDecimal percent) {
    BigDecimal result =
amount.multiply(percent).divide(BigDecimal.valueOf(100),
currency.getDefaultFractionDigits(), RoundingMode.HALF_UP);
    return new Money(result, currency);
}
```

Выводы по заданию:

Использование Money предотвращает ошибки некорректных вычислений и случайностей с разделителями валют.

5.3 Задание 15. Расписание и цена вокруг границ времени

Постановка задачи:

Реализовать сквозную модель CourseHub для вебинаров: временные зоны, дедлайны скидок early-bird и расчет возврата средств.

Анализ решения и теоретическое обоснование:

График вебинаров строится на Instant с календарным шагом Period.ofWeeks(1) в часовом поясе организатора. Скидка early-bird активна строго до момента deadline. Возврат средств при отмене рассчитывается пропорционально количеству оставшихся занятий.

Код решения:

Листинг 15 — *Служба расчета возврата и цен*

```
public Money refund(Purchase purchase, Instant moment) {
    int total = purchase.offering().plan().sessionCount();
    int remaining =
purchase.offering().plan().remainingSessions(moment);
    if (remaining <= 0) return Money.of("0",
purchase.paid().currency());
    BigDecimal percent = BigDecimal.valueOf(remaining)
        .multiply(BigDecimal.valueOf(100))
        .divide(BigDecimal.valueOf(total), 6, RoundingMode.HALF_UP);
```

```
    return purchase.paid().percentage(percent);
}
```

Выводы по заданию:

Создана устойчивая доменная модель, корректно обрабатывающая DST, високосные дни, часовые пояса и денежные расчеты.

6. Заключение

В ходе выполнения практической работы были достигнуты следующие результаты:

1. Изучены и применены на практике механизмы работы со строками (String, StringBuilder, Pattern/Matcher), включая правила эффективной конкатенации, канонизацию в string pool и регулярные выражения с группами.
2. Освоены точные вычисления с BigDecimal, предотвращающие ошибки представления вещественных чисел double, правила работы с масштабом, детерминированное распределение остатков и безопасное использование автоупаковки Integer.
3. Изучен современный API даты и времени Java (LocalDate, Instant, ZonedDateTime, Period, Duration), реализована обработка переходов на летнее/зимнее время (DST) и созданы строгие форматировщики дат.
4. Спроектирована и реализована сквозная доменная модель учебного центра CourseHub, объединяющая поиск курсов, неизменяемые денежные расчеты в классе Money и гибкий расчет стоимости и возвратов средств за вебинары.

Приложение А. Полные листинги программы

Листинг А.1 (Общие данные) — *Файл сборки pract6/pom.xml*

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
 <modelVersion>4.0.0</modelVersion>
 <groupId>com.example</groupId>
 <artifactId>coursehub-strings-numbers-time</artifactId>
 <version>1.0-SNAPSHOT</version>
 <properties>
 <maven.compiler.release>21</maven.compiler.release>
 <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
 </properties>
</project>
```

Листинг А.2 (Задание 1) — Класс StringIdentityDemo.java

```
package coursehub.practice6.strings;

public class StringIdentityDemo {
 public static void main(String[] args) {
 String a = "JAVA-101";
 String b = "JAVA-101";
 System.out.println("literal == literal: " + (a == b));
 System.out.println("literal.equals(literal): " + a.equals(b));
 String c = new String("JAVA-101");
 System.out.println("literal == new String: " + (a == c));
 System.out.println("literal.equals(new String): " + a.equals(c));
 String d = "JAVA" + "-101";
 System.out.println("literal == compile-time concat: " + (a == d));
 String prefix = "JAVA";
 String e = prefix + "-101";
 System.out.println("literal == runtime concat: " + (a == e));
 System.out.println("literal == interned: " + (a == e.intern()));
 }
}
```

Листинг А.3 (Задание 2) — Класс ReportBuilder.java

```
package coursehub.practice6.strings;

import java.util.List;

public final class ReportBuilder {
 private ReportBuilder() {}
```

```
 public static String withPlus(List<String> lines) {
 String report = "";
 for (String line : lines) {
 report += line + "\n";
 }
 return report;
 }

 public static String withBuilder(List<String> lines) {
 StringBuilder sb = new StringBuilder();
 for (String line : lines) {
 sb.append(line).append('\n');
 }
 return sb.toString();
 }
}
```

Листинг A.4 (Задание 3) — Класс CourseCode.java

```
package coursehub.practice6.strings;

import java.util.Optional;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public record CourseCode(String prefix, int number) {
 private static final Pattern FORMAT = Pattern.compile("\\A([A-Z]+)-
(\\d+)\\z");

 public static Optional<CourseCode> parse(String raw) {
 if (raw == null) return Optional.empty();
 Matcher matcher = FORMAT.matcher(raw);
 if (!matcher.matches()) return Optional.empty();
 return Optional.of(new CourseCode(matcher.group(1),
Integer.parseInt(matcher.group(2))));
 }
}
```

Листинг A.5 (Задание 4) — Класс IntegerPolicy.java

```
package coursehub.practice6.numbers;

public final class IntegerPolicy {
 private IntegerPolicy() {}

 public static int plusOneUnsafe(Integer value) {
 return value + 1;
 }
```

```
 public static boolean sameByReference(Integer left, Integer right) {
 return left == right;
 }

 public static int plusOne(Integer value) {
 return value == null ? 0 : value + 1;
 }

 public static boolean sameByValue(Integer left, Integer right) {
 return java.util.Objects.equals(left, right);
 }
}
```

Листинг A.6 (Задание 5) — Класс DecimalDemo.java

```
package coursehub.practice6.numbers;

import java.math.BigDecimal;

public class DecimalDemo {
 public static void main(String[] args) {
 BigDecimal fromDouble = new BigDecimal(0.1);
 BigDecimal valueOf = BigDecimal.valueOf(0.1);
 BigDecimal fromString = new BigDecimal("0.1");

 System.out.println("new BigDecimal(0.1): " + fromDouble);
 System.out.println("BigDecimal.valueOf(0.1): " + valueOf);
 System.out.println("new BigDecimal(\"0.1\"): " + fromString);

 System.out.println("valueOf.equals(string): " +
valueOf.equals(fromString));
 System.out.println("fromDouble.equals(string): " +
fromDouble.equals(fromString));
 }
}
```

Листинг A.7 (Задание 6) — Класс DiscountAllocator.java

```
package coursehub.practice6.numbers;

import java.math.BigDecimal;
import java.math.RoundingMode;

public final class DiscountAllocator {
 private static final int SCALE = 2;
 private static final BigDecimal CENT = new BigDecimal("0.01");

 private DiscountAllocator() {}
```

```
 public static BigDecimal[] allocate(BigDecimal amount, int[] weights,
RoundingMode mode) {
 int totalWeight = 0;
 for (int weight : weights) totalWeight += weight;
 BigDecimal[] shares = new BigDecimal[weights.length];
 BigDecimal distributed = BigDecimal.ZERO.setScale(SCALE);
 for (int i = 0; i < weights.length; i++) {
 BigDecimal share =
amount.multiply(BigDecimal.valueOf(weights[i])).divide(BigDecimal.valueO
f(totalWeight), SCALE, mode);
 shares[i] = share;
 distributed = distributed.add(share);
 }
 BigDecimal remainder = amount.setScale(SCALE,
mode).subtract(distributed);
 int index = 0;
 while (remainder.signum() != 0) {
 BigDecimal step = remainder.signum() > 0 ? CENT : CENT.negate();
 shares[index] = shares[index].add(step);
 remainder = remainder.subtract(step);
 index = (index + 1) % shares.length;
 }
 return shares;
 }
}
```

Листинг A.8 (Задание 7) — Класс RandomSampleService.java

```
package coursehub.practice6.numbers;

import java.util.random.RandomGenerator;
import java.util.random.RandomGeneratorFactory;

public final class RandomSampleService {
 private static final String ALGORITHM = "L64X128MixRandom";
 private RandomSampleService() {}

 public static RandomGenerator production() {
 return RandomGeneratorFactory.of(ALGORITHM).create();
 }

 public static RandomGenerator seeded(long seed) {
 return RandomGeneratorFactory.of(ALGORITHM).create(seed);
 }
}
```

Листинг A.9 (Задание 8) — Класс StudyCalendar.java

```
package coursehub.practice6.time;

import java.time.LocalDate;
import java.time.Year;
import java.util.ArrayList;
import java.util.List;

public final class StudyCalendar {
 private final LocalDate start;
 public StudyCalendar(LocalDate start) {
 if (start == null) throw new IllegalArgumentException("start must not
be null");
 this.start = start;
 }
 public LocalDate start() { return start; }
 public boolean isLeapYear() { return Year.isLeap(start.getYear()); }
 public List<LocalDate> moduleStarts(int modules, int moduleWeeks) {
 List<LocalDate> starts = new ArrayList<>();
 LocalDate date = start;
 for (int i = 0; i < modules; i++) {
 starts.add(date);
 date = date.plusWeeks(moduleWeeks);
 }
 return starts;
 }
}
```

Листинг A.10 (Задания 9, 11) — Класс WebinarSchedule.java

```
package coursehub.practice6.time;

import java.time.Duration;
import java.time.Instant;
import java.time.Period;
import java.time.ZoneId;
import java.time.ZonedDateTime;

public final class WebinarSchedule {
 private final Instant start;
 public WebinarSchedule(Instant start) {
 if (start == null) throw new IllegalArgumentException("start must not
be null");
 this.start = start;
 }
 public Instant start() { return start; }
 public ZonedDateTime plusCalendarDays(int days, ZoneId zone) { return
start.atZone(zone).plus(Period.ofDays(days)); }
 public Instant plusExactHours(long hours) { return
```

```
start.plus(Duration.ofHours(hours)); }
 public ZonedDateTime displayIn(ZoneId zone) { return
start.atZone(zone); }
}
```

Листинг A.11 (Задание 10) — Класс DateParser.java

```
package coursehub.practice6.time;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.time.format.ResolverStyle;

public final class DateParser {
 private static final DateTimeFormatter FORMAT =
DateTimeFormatter.ofPattern("dd.MM.uuuu").withResolverStyle(ResolverStyl
e.STRICT);
 private DateParser() {}
 public static LocalDate parse(String raw) {
 try {
 return LocalDate.parse(raw, FORMAT);
 } catch (DateTimeParseException e) {
 throw new InvalidDateFieldException("date", raw, e);
 }
 }
}
```

Листинг A.12 (Задание 12) — Класс LocalDateTimeResolver.java

```
package coursehub.practice6.time;

import java.time.LocalDateTime;
import java.time.ZoneId;
import java.time.ZonedDateTime;

public final class LocalDateTimeResolver {
 private LocalDateTimeResolver() {}
 public static ZonedDateTime resolveGapForward(LocalDateTime local,
ZoneId zone) { return local.atZone(zone); }
 public static ZonedDateTime resolveStrict(LocalDateTime local, ZoneId
zone) {
 ZonedDateTime resolved = local.atZone(zone);
 if (!resolved.toLocalDateTime().equals(local)) throw new
InvalidLocalTimeException(local, zone);
 return resolved;
 }
}
```

Листинг A.13 (Задания 13, 15) — Класс CourseTitle.java

```
package coursehub.practice6.domain;

import java.text.Normalizer;
import java.util.Locale;

public record CourseTitle(String display, String searchKey) {
 public CourseTitle(String display) { this(display,
searchKeyOf(display)); }
 public static String searchKeyOf(String raw) {
 if (raw == null) throw new IllegalArgumentException("title must not be
null");
 return Normalizer.normalize(raw,
Normalizer.Form.NFKC).strip().replaceAll("\\s+", "
").toLowerCase(Locale.ROOT);
 }
}
```

Листинг A.14 (Задания 14, 15) — Класс Money.java

```
package coursehub.practice6.domain;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.Currency;
import java.util.Objects;

public final class Money {
 private final BigDecimal amount;
 private final Currency currency;
 private Money(BigDecimal amount, Currency currency) {
 this.amount = amount.setScale(currency.getDefaultFractionDigits(),
RoundingMode.HALF_UP);
 this.currency = currency;
 }
 public static Money of(String amount, Currency currency) { return new
Money(new BigDecimal(amount), currency); }
 public Money add(Money other) { requireSameCurrency(other); return new
Money(amount.add(other.amount), currency); }
 private void requireSameCurrency(Money other) {
 if (!currency.equals(other.currency)) throw new
IllegalArgumentException("currency mismatch");
 }
 public Money percentage(BigDecimal percent) {
 return new
Money(amount.multiply(percent).divide(BigDecimal.valueOf(100),
currency.getDefaultFractionDigits(), RoundingMode.HALF_UP), currency);
```

```
 }
}
```

Листинг A.15 (Задание 15) — Класс WebinarSeriesPlan.java

```
package coursehub.practice6.domain;

import java.time.Duration;
import java.time.Instant;
import java.time.Period;
import java.time.ZoneId;
import java.time.ZonedDateTime;
import java.util.ArrayList;
import java.util.List;

public record WebinarSeriesPlan(CourseTitle title, Instant firstSession,
ZoneId organizerZone, int sessionCount, Duration sessionDuration) {
 public List<Instant> sessionStarts() {
 List<Instant> starts = new ArrayList<>();
 ZonedDateTime session = firstSession.atZone(organizerZone);
 for (int i = 0; i < sessionCount; i++) {
 starts.add(session.toInstant());
 session = session.plus(Period.ofWeeks(1));
 }
 return starts;
 }
 public int remainingSessions(Instant moment) {
 int remaining = 0;
 for (Instant start : sessionStarts()) if (start.isAfter(moment))
remaining++;
 return remaining;
 }
}
```

Листинг A.16 (Задание 15) — Класс PricingService.java

```
package coursehub.practice6.domain;

import java.time.Instant;

public final class PricingService {
 private final EarlyBirdPolicy policy;
 public PricingService(EarlyBirdPolicy policy) { this.policy = policy; }
 public Purchase purchase(CourseOffering offering, Instant moment) {
 Money price = policy.priceFor(offering, moment);
 return new Purchase(offering, price, moment);
 }
}
```

Листинг A.17 (Задание 15) — Класс RefundService.java

```
package coursehub.practice6.domain;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.Instant;

public final class RefundService {
 public Money refund(Purchase purchase, Instant moment) {
 WebinarSeriesPlan plan = purchase.offering().plan();
 int total = plan.sessionCount();
 int remaining = plan.remainingSessions(moment);
 if (remaining <= 0) return Money.of("0", purchase.paid().currency());
 BigDecimal percent = BigDecimal.valueOf(remaining)
 .multiply(BigDecimal.valueOf(100))
 .divide(BigDecimal.valueOf(total), 6, RoundingMode.HALF_UP);
 return purchase.paid().percentage(percent);
 }
}
```